

ADVENTIST UNIVERSITY OF
CENTRAL AFRICA

BIG DATA ANALYTICS

Distributed Multi-Model Analytics for E-commerce Data

Final Project Technical Report_{1cm}

Prepared by: Richard Mwenedata

Student ID: 101215

Program: Master of Science in Big data analytics

Course: Big Data Analytics

Instructor: Temitope Egutande

This project can be found on GitHub, [click this link](#)

Technologies Used

MongoDB • HBase • Apache Spark • Python

25 June, 2026

Kigali, Rwanda

Contents

1	System Architecture Overview	2
1.1	Architecture Components	2
1.2	Data flow	3
2	Data Modeling Decisions	3
2.1	MongoDB Schema Design	3
2.1.1	Collections and Justification	3
2.1.2	Indexes	4
2.1.3	Aggregation pipelines	4
2.2	HBase schema design	5
2.2.1	Table: <code>user_sessions</code>	5
2.2.2	Table: <code>product_metrics</code>	6
2.3	MongoDB vs HBase: Comparison	7
3	Spark Processing Pipeline	8
3.1	Data Cleaning and Normalization	8
3.2	Batch Job 1: Product Co-Purchase Analysis	8
3.3	Batch Job 2: Cohort Analysis	9
3.4	Spark SQL Analytics	9
3.5	Optimization Choices	12
4	Integrated Analytics Workflows	12
4.1	Customer Lifetime Value (CLV) Estimation	12
4.2	Product Affinity and Recommendation	13
4.3	Funnel Conversion Analysis	14
4.4	Seasonal Trend Identification	15
5	Key Findings and Business Insights	17
5.1	Revenue and Sales Performance	17
5.2	Customer Behavior	18
5.3	Product Insights	19
5.4	Customer Lifetime Value	20
6	Scalability Considerations	21
6.1	MongoDB Scalability	21
6.2	HBase Scalability	21
6.3	Apache Spark Scalability	21
6.4	Data volume projections	22
7	Technology Selection Justification	22
8	Limitations and Future Work	24
8.1	Current Limitations	24
8.2	Future Work	24
9	Conclusion	25

1 System Architecture Overview

This project implements a distributed multi-model analytics platform for large-scale e-commerce data, combining three complementary technologies: MongoDB for document-oriented storage, HBase for time-series wide-column storage, and Apache Spark for distributed batch processing. Each technology is selected based on its inherent strengths relative to the data characteristics and analytical requirements of the use case.

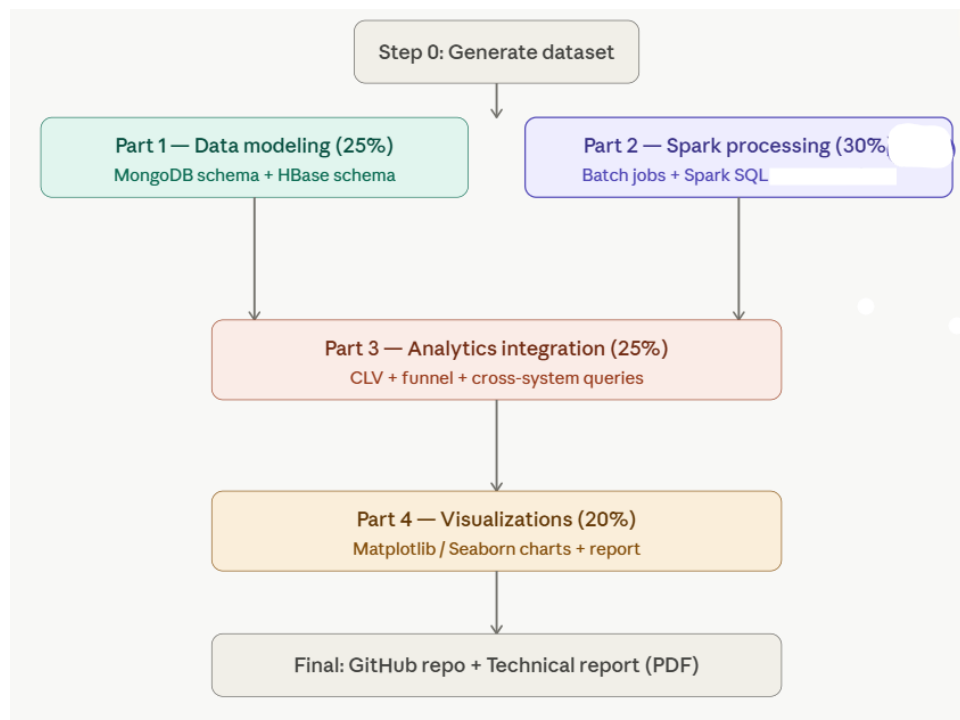


Figure 1: System Architecture

1.1 Architecture Components

Component	Role	Data Stored	Access Pattern
MongoDB	Document store	Users, Products, Transactions, Categories	Rich queries, aggregation pipelines
HBase	Wide-column store	User sessions (time-series), Product daily metrics	Row key prefix scans, time-range reads
Apache Spark	Distributed processing	All data sources (reads JSON / connects via connectors)	Batch jobs, SQL analytics, cross-system joins
Python / PySpark	Orchestration layer	Scripts, visualizations, data generation	ETL, ML preparation, reporting

Table 1: Multi-Model Architecture Components

1.2 Data flow

Raw JSON files generated by the dataset generator serve as the input layer. MongoDB ingests users, products, transactions, and categories. HBase stores session time-series data using a reverse-timestamp row key design for efficient recency first retrieval. Apache Spark reads all sources either directly from JSON (simulating connector reads) or via native connectors and performs cross-system analytical joins. Final outputs are CSV files consumed by the visualization layer to produce business insights

2 Data Modeling Decisions

2.1 MongoDB Schema Design

MongoDB was selected for entities that are document-oriented, have nested structure, and require rich query capabilities. Four s were designed:

2.1.1 Collections and Justification

Collection	Key Design Decision	Justification
users	Flat profile with geo_data sub-document	User profiles are read as atomic units; embedding geo avoids joins on the hot path.
products	Embedded price_history array	Price changes are always read with the product; embedding avoids a separate collection lookup.
transactions	Embedded items[] array	A transaction and its line items are always written and read together as an atomic document.
categories	Nested subcategories[] array	Category hierarchy is rarely updated and always read as a complete tree.

Table 2: MongoDB collection design decisions

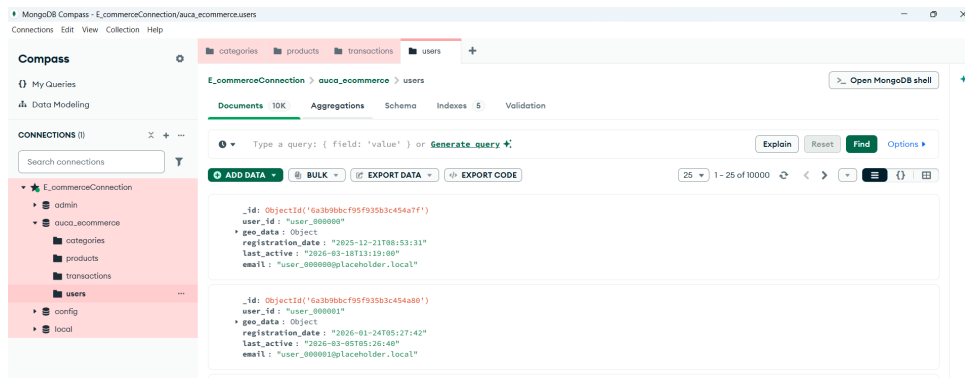


Figure 2: collections in mongoCompass

2.1.2 Indexes

Strategic indexes were created to support the aggregation pipelines: unique index on user-id and email (sparse=True to handle nulls), index on category-id for product lookups, compound index on (timestamp DESC) for transaction time-range queries, and index on status for filtering completed orders.

2.1.3 Aggregation pipelines

Three aggregation pipelines were implemented to analyze different aspects of the e-commerce data stored in MongoDB.

- **Pipeline A: Product Popularity** This pipeline unwinds the transaction items, groups them by `product_id`, performs a lookup to retrieve product names from the `products` collection, and sorts the results by total revenue. It identifies the top-selling products while also calculating the average selling price compared with the original base price.

```
[Pipeline A] Product popularity (top 15 by revenue)...
Query time: 65.4ms
```

Product	Category	Units	Revenue	Avg Price
Business-Focused Dedicated Encodin	cat_011	25	\$12,186.25	\$484.25
De-Engineered Demand-Driven Functi	cat_013	15	\$8,750.25	\$583.35
Innovative Non-Volatile Capacity	cat_012	19	\$8,290.88	\$436.32
Monitored Zero Tolerance Initiativ	cat_024	14	\$8,269.94	\$590.71
Robust Demand-Driven Parallelism	cat_017	23	\$8,042.64	\$349.68
Persevering Composite Process Impr	cat_010	15	\$7,850.55	\$523.37
Open-Source Hybrid Complexity	cat_022	17	\$7,846.35	\$461.55
Innovative Methodical Synergy	cat_002	13	\$7,822.23	\$601.71
Optimized Regional Process Improve	cat_013	15	\$7,686.98	\$512.46
Closed Optimizing Capacity	cat_001	22	\$7,667.88	\$348.54
Persistent Encompassing Productivi	cat_005	15	\$7,489.65	\$499.31
Versatile Bifurcated Application	cat_017	17	\$7,388.98	\$429.94
Centralized 4Thgeneration Parallel	cat_007	16	\$6,678.56	\$417.41
Open-Source Even-Keeled Capability	cat_020	18	\$6,628.32	\$368.24
Persevering Grid-Enabled Standardi	cat_018	14	\$6,580.98	\$470.07

Figure 3: Product popularity

- **Pipeline B: User Segmentation** This pipeline aggregates total spending for each user, joins the results with user profile information to obtain attributes such as age and payment preferences, groups users into different age categories, and computes key metrics including average order value and the percentage of high-value customers.

```
[Pipeline B] User segmentation by age group & spend...
```

Age Group	Users	Avg Orders	Avg Spend	Avg Order \$	High Value%
55+	127	2.3	\$ 2,697.65	\$ 1,153.58	78.7%
25-34	72	1.8	\$ 2,492.37	\$ 1,289.28	72.2%
18-24	52	1.9	\$ 2,470.26	\$ 1,309.43	82.7%
45-54	90	2.1	\$ 2,361.76	\$ 1,145.34	77.8%
35-44	70	2.0	\$ 2,237.77	\$ 1,066.77	71.4%

Figure 4: User segmentation

- **Pipeline C: Revenue by Category** This pipeline unwinds transaction items, joins them with the `products` collection to obtain product category information, and aggregates total revenue, units sold, and the number of transactions for each product category.

```
[Pipeline C] Revenue by category...
```

Category	Revenue	Units	Transactions
cat_002	\$ 62,049.88	203	79
cat_016	\$ 53,795.36	172	66
cat_003	\$ 53,701.20	208	81
cat_018	\$ 53,652.11	171	68
cat_017	\$ 52,573.48	197	75
cat_004	\$ 51,410.79	176	77
cat_012	\$ 50,652.37	175	67
cat_021	\$ 48,868.17	175	72
cat_020	\$ 47,581.91	215	86
cat_001	\$ 46,923.10	161	65

Figure 5: Revenue by category

2.2 HBase schema design

HBase was chosen for session data because sessions are write-heavy, time-series in nature, and accessed almost exclusively by user ID and time range a pattern perfectly suited to HBase’s row key design. The file called `part1_habase.py` (in project folder) has all the details.

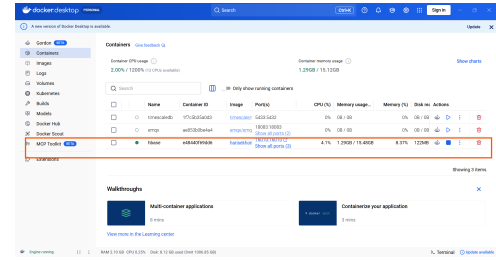


Figure 6: Dockerlised HBase

2.2.1 Table: `user_sessions`

Design Element	Value	Rationale
Row Key	<code>user_id</code> <code>reverse_timestamp</code>	The reverse timestamp ensures that the most recent sessions are sorted first, while the <code>user_id</code> prefix co-locates all sessions belonging to the same user for efficient retrieval.
Column Family: info	<code>start_time</code> , <code>end_time</code> , <code>duration</code> , <code>device</code> , <code>referrer</code> , <code>conversion</code>	Stores frequently accessed session meta-data. This column family is designed for regular queries and is configured with a Time-To-Live (TTL) of forever.
Column Family: activity	<code>viewed_products</code> , <code>cart</code> , <code>page_count</code>	Stores user behavioral data. Since many sessions may not contain shopping cart information, this column family benefits from HBase’s efficient handling of sparse data.
Column Family: geo	<code>city</code> , <code>state</code> , <code>ip</code>	Stores geographic information that is accessed less frequently. Keeping it in a separate column family reduces unnecessary data loading during common read operations.

Table 3: HBase Design for the `user_sessions` Table

TABLE 1: user_sessions	

Purpose: Store user browsing sessions as a time-series stream.	
Row key: {user_id}#{reverse_timestamp}	
-	Reverse timestamp = (Long.MAX_VALUE - epoch_ms) so the most recent session for a user sorts FIRST (HBase stores rows in lexicographic ascending order by row key).
-	Prefix on user_id → all sessions for one user are co-located on the same region, enabling fast single-user range scans.
-	Example: user_000042#9223370799053008807
Column families:	
info:	(TTL=forever, VERSIONS=1)
info:start_time	- ISO timestamp
info:end_time	- ISO timestamp
info:duration_secs	- integer
info:device_type	- mobile/desktop/tablet
info:device_os	- iOS/Android/Windows...
info:referrer	- search_engine/social_media/...
info:conversion	- converted/abandoned
activity:	(TTL=forever, VERSIONS=1)
activity:viewed	- JSON array of product_ids
activity:cart	- JSON map of {product_id: qty}
activity:page_count	- integer
geo:	(TTL=forever, VERSIONS=1)
geo:city	- city name
geo:state	- state abbreviation
geo:ip	- IP address
Why HBase vs MongoDB for sessions:	
•	Sessions are WRITE-HEAVY and HIGH-VOLUME (millions/day at scale).
•	Access pattern is almost always: "give me user X's sessions between time A and time B" – perfect for row key prefix scan.
•	Many columns may be empty (cart is empty for non-converters) – HBase's sparse wide-column model doesn't waste space on nulls.
•	MongoDB would need to scan many documents or maintain large user-embedded arrays; HBase row key scans are O(1) seek + O(n) read.

Figure 7: User sessions details outputs

2.2.2 Table: product_metrics

- **Row key:** product-id date (YYYY-MM-DD).
Column family: daily with views, cart-adds, purchases, revenue. Enables efficient time-range scans for any product's performance over any date window. Native HBase increment operations allow real-time counter updates without read-modify-write cycles.

TABLE 2: product_metrics	

Purpose: Track product view and purchase counts per day.	
Row key: {product_id}#{date}	
-	Product-first prefix allows scanning all days for one product.
-	Date in YYYY-MM-DD format for natural chronological order.
-	Example: prod_00123#2025-01-15
Column families:	
daily:	(TTL=90days, VERSIONS=1)
daily:views	- total page views that day
daily:cart_adds	- added to cart count
daily:purchases	- units purchased
daily:revenue	- revenue generated
Why HBase for product metrics:	
•	Time-series incremental counters → HBase has native increment operations (table.incrementColumnValue).
•	Can scan prod_00123 from 2025-01-01 to 2025-03-31 with a single efficient range scan.
•	Would be expensive in MongoDB (many small documents or large embedded arrays updated on every purchase).

Figure 8: product metrics outputs

Design Element	Value	Rationale
Row Key	product_id#date (YYYY-MM-DD)	Combines product identifier with date to support efficient time-series queries and range scans over specific time windows.
Column Family: daily	views, cart_adds, purchases, revenue	Stores daily aggregated metrics for each product. Enables fast updates and retrieval of performance indicators over time.
Update Mechanism	HBase increment operation	Allows real-time counter updates without requiring expensive read-modify-write cycles, making it suitable for high-throughput analytics workloads.

Table 4: HBase Design for the `product_metrics` Table

2.3 MongoDB vs HBase: Comparison

Criterion	MongoDB	HBase
Data model	Flexible documents with nested structures	Sparse wide-column rows optimized for scale
Query style	Rich ad-hoc queries and powerful aggregations	Row-key scans and efficient range reads
Write pattern	Moderate write volume with structured updates	Very high write throughput, append-heavy workloads
Best for	Users, products, and transactional data	Sessions and time-series metrics
Weakness	Performance degradation at very high write scale	No native support for ad-hoc querying

Table 5: MongoDB vs HBase comparison

3 Spark Processing Pipeline

3.1 Data Cleaning and Normalization

The cleaning pipeline handles several real-world data quality issues present in the generated dataset. Missing columns (email, age, gender) are detected at runtime and filled with safe defaults rather than crashing. Null emails are replaced with placeholder values to maintain referential integrity. Timestamps are parsed from ISO 8601 strings to native Spark TimestampType. Non-positive durations in sessions are filtered out. Categorical fields like payment_method are lowercased and trimmed for consistency. Derived columns are added during cleaning: age_group bins users into five demographic bands (18-24, 25-34, 35-44, 45-54, 55+), reg_month extracts the registration month for cohort analysis, price_tier classifies products into budget/mid-range/premium/luxury, and txn_week uses ISO week-year format (YYYY-ww) for weekly trend analysis.

```
PS D:\AUCA ARCHIVES\MASTERS\BIG DATA ANALYTICS\Final_Project> py part2_spark_processing.py

WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/06/27 13:47:25 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

=====
AUCA Big Data - Part 2: Spark Processing
=====

[2.1] Loading and cleaning raw JSON data...
Users: 500 rows after cleaning
Products: 500 rows after cleaning
Transactions: 1078 rows after cleaning
26/06/27 13:48:07 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
Sessions: 1505000 rows across 20 files

✓ All data cleaned and registered as Spark SQL views
```

Figure 9: Cleaned data

3.2 Batch Job 1: Product Co-Purchase Analysis

Business question: Which products are most frequently bought together, enabling cross-sell recommendations? The pipeline explodes transaction items to get one row per (transaction, product), then performs a self-join on transaction_id to create all product pairs purchased in the same order. Pairs where product_a < product_b are kept to avoid duplicates and self-pairs. The result is grouped by product pair and counted, then enriched with product names and categories. The top 20 co-purchased pairs are shown, with the full top-200 saved for the recommendation engine.

Top 10 co-purchased product pairs:				
product_a	product_a_name	product_b	product_b_name	co_purchase_count
prod_00161	Triple-Buffered Methodical Hierarchy	prod_00416	Reverse-Engineered Responsive Groupware	2
prod_00057	Programmable Local Installation	prod_00287	Proactive Multi-Tasking Product	2
prod_00128	Managed Asymmetric Parallelism	prod_00208	Persistent Encompassing Productivity	2
prod_00162	Enterprise-Wide Heuristic Implementation	prod_00442	Virtual High-Level Analyzer	2
prod_00255	Optimized User-Facing Policy	prod_00443	Enhanced Directional Firmware	2
prod_00252	Front-Line Regional Knowledge User	prod_00479	Organized Scalable Framework	2
prod_00410	Open-Source Even-Keeled Capability	prod_00412	Public-Key Local Functionalities	1
prod_00355	Diverse Client-Server Middleware	prod_00365	Sharable Logistical Task-Force	1
prod_00197	Cross-Platform System-Worthy Service-Desk	prod_00199	Optional Dedicated Methodology	1
prod_00197	Cross-Platform System-Worthy Service-Desk	prod_00333	Intuitive Impactful Throughput	1

Figure 10: users who bought X also bought Y” based on transaction data

3.3 Batch Job 2: Cohort Analysis

Business Question: How does spending evolve over time for users who registered in the same month? Users are grouped by their registration month (cohort). Each transaction is joined with the user's registration cohort, and `months_since_registration` is computed using `months_between()`. The result is a cohort $\times$ time-offset matrix showing total revenue, average order value, and number of active users per cohort per period. A pivot table summarizes revenue for the first three months post-registration.

Cohort spending (first 15 rows):

reg_month	months_since_reg	num_transactions	total_revenue	avg_order_value	active_users
2024-10	3	135	164943.91999999995	1221.8068148148145	91
2024-10	4	106	128739.16999999997	1214.5204716981113	77
2024-10	5	116	145929.43	1258.0123275862068	85
2024-11	2	118	137763.28000000001	1167.4854237288143	82
2024-11	3	116	128417.40000000002	1107.0465517241382	84
2024-11	4	121	148743.50000000003	1229.2851239669424	88
2024-12	1	123	141867.62	1153.3952845528454	87
2024-12	2	122	159795.80000000008	1309.8016393442629	87
2024-12	3	121	140934.06999999998	1164.7443801652892	84

Figure 11: Cohort analysis: spending by registration month..

3.4 Spark SQL Analytics

Six analytical queries were implemented using Spark SQL and the DataFrame API:

Query	Business Question	Key Technique
Revenue by Category	Which categories generate the most revenue?	DataFrame <code>explode</code> , join, and <code>groupBy</code> aggregation
Conversion Funnel	How many sessions convert at each funnel stage?	Conditional aggregation with type casting
Product Performance	Which products have the best view-to-purchase ratio?	Left join between views and purchases DataFrames
User Segmentation	How do spending patterns vary by age and payment method?	Join users and transactions; CASE logic for customer lifetime value (CLV) tiering
Weekly Revenue Trend	Is revenue growing week over week?	Window function with 4-week rolling average
Device & Referrer Stats	Which devices and referrers convert best?	Grouped aggregation with conditional sums

Table 6: Summary of Analytical Queries and Techniques

Revenue by category (top 10):

category_id	num_transactions	num_customers	total_revenue	avg_item_revenue
cat_002	103	91	78886.53	758.52
cat_004	104	92	77208.55	701.9
cat_016	81	74	69951.46	842.79
cat_018	81	74	66863.75	805.59
cat_021	91	81	65537.75	704.71
cat_012	84	76	64725.75	752.63
cat_017	85	79	62294.86	707.9
cat_020	104	95	59916.35	549.69
cat_003	89	84	59306.81	644.64
cat_011	76	71	56943.84	749.26

only showing top 10 rows

Figure 12: Revenue by category

Conversion funnel:

total_sessions	viewed_product	added_to_cart	converted	conversion_rate_pct
1505000	1486473	1486473	305892	20.33

Figure 13: Conversion funnel

Product performance (top 10 by revenue):

name	category_id	total_views	units_sold	revenue	view_to_purchase_rate
Business-Focused Dedicated Encoding	cat_011	141	25	12106.25	0.1773
De-Engineered Demand-Driven Function	cat_013	879	15	8750.25	0.0171
Innovative Non-Volatile Capacity	cat_012	2063	19	8290.08	0.0092
Monitored Zero Tolerance Initiative	cat_024	2056	14	8269.94	0.0068
Robust Demand-Driven Parallelism	cat_017	1981	23	8042.64	0.0116
Persevering Composite Process Improve...	cat_010	1993	15	7850.55	0.0075
Open-Source Hybrid Complexity	cat_022	2023	17	7846.35	0.0084
Innovative Methodical Synergy	cat_002	2061	13	7822.23	0.0063
Optimized Regional Process Improvement	cat_013	1994	15	7686.9	0.0075
Cloned Optimizing Capacity	cat_001	178	22	7667.88	0.1236

only showing top 10 rows

Figure 14: Product performance

User segments (sample 10):

user_id	age_group	gender	preferred_payment	num_orders	lifetime_value	avg_order_value	max_order_value	first_purchase	last_purchase	customer_segment
user_000010	55+	F	credit_card	2	1813.47	906.74	1017.03	2025-02-12	2025-03-03	high_value
user_000018	35-44	F	debit_card	3	4185.36	1395.12	1985.16	2025-01-05	2025-03-28	high_value
user_000021	25-34	Other	paypal	3	3863.06	1287.69	2730.51	2025-01-02	2025-03-19	high_value
user_000029	55+	Other	credit_card	2	4334.16	2167.08	2900.66	2025-01-22	2025-02-23	high_value
user_000033	55+	F	debit_card	3	1672.76	557.59	1137.51	2025-01-16	2025-02-25	high_value
user_000034	55+	M	credit_card	4	7298.58	1824.65	2888.79	2025-01-31	2025-03-13	high_value
user_000037	45-54	M	credit_card	1	1270.68	1270.68	1270.68	2025-01-13	2025-01-13	high_value
user_000042	35-44	Other	paypal	3	1620.14	540.05	1271.28	2025-01-14	2025-03-24	high_value
user_000049	45-54	F	debit_card	2	3408.99	1704.5	3133.23	2025-02-26	2025-03-10	high_value
user_000064	55+	F	debit_card	2	824.79	412.4	519.98	2025-02-20	2025-03-09	mid_value

only showing top 10 rows

Figure 15: User segmentation

Weekly revenue trend:				
txn_week	num_transactions	total_revenue	avg_order_value	unique_customers
2025-01	38	42253.29	1111.93	38
2025-02	69	77373.81	1121.36	64
2025-03	74	85351.66	1153.4	68
2025-04	61	76664.8	1256.8	55
2025-05	71	83591.06	1177.34	67
2025-06	59	66014.2	1118.88	54
2025-07	75	92337.74	1231.17	65
2025-08	72	86666.97	1203.71	69
2025-09	72	80881.91	1123.36	68
2025-10	70	92244.46	1317.78	65
2025-11	69	81999.64	1188.4	68
2025-12	58	70246.69	1211.15	55
2025-13	65	77271.37	1188.79	62
2025-14	5	6809.15	1361.83	5

Figure 16: Weekly revenue trend

Conversion rate by device & referrer:					
device_type	device_os	referrer	num_sessions	conversions	conv_rate_pct
tablet	Android	email	25420	5293	20.82
tablet	Android	search_engine	25326	5048	19.93
mobile	macOS	affiliate	25307	5143	20.32
mobile	iOS	email	25299	5082	20.09
mobile	Android	search_engine	25281	5024	19.87
tablet	macOS	social	25207	5160	20.47
desktop	Windows	direct	25196	5168	20.51
mobile	Windows	social	25196	5136	20.38
desktop	macOS	social	25156	5153	20.48
mobile	Windows	affiliate	25154	5104	20.29
desktop	macOS	affiliate	25150	5058	20.11
mobile	Android	social	25149	5082	20.21
desktop	macOS	email	25147	5113	20.33
mobile	Android	affiliate	25135	5150	20.49
tablet	Android	direct	25129	5142	20.46
desktop	Windows	affiliate	25115	5013	19.96
mobile	iOS	search_engine	25113	5152	20.52
tablet	iOS	affiliate	25112	5077	20.22
tablet	Android	affiliate	25109	5126	20.41
mobile	iOS	direct	25106	5164	20.57

only showing top 20 rows

Figure 17: Device and referrer statistics .

3.5 Optimization Choices

Several optimizations were applied to support efficient local execution on a Windows environment.

- **Shuffle Optimization:** Spark shuffle partitions were reduced from the default value of 200 to 8, which is more suitable for small-scale local datasets and significantly reduces shuffle overhead.
- **Time Parsing Configuration:** The parameter `spark.sql.legacy.timeParserPolicy` was set to `LEGACY` to ensure consistent handling of ISO week formatting across different Spark versions.
- **Windows Compatibility Fix:** To avoid Hadoop dependency issues on Windows (e.g., `winutils.exe` and `UnsatisfiedLinkError`), CSV exports were handled using `toPandas().to_csv()` instead of Spark's native write function.
- **Memory Configuration:** Driver memory was set to 4GB, and `spark.driver.maxResultSize` was configured to 2GB to support large DataFrame operations, particularly during co-purchase self-join processing.
- **Early Column Pruning:** Heavy nested columns such as `cart_contents` (STRUCT) and `page_views` (ARRAY) were dropped early in the pipeline to reduce memory usage and improve execution efficiency.

4 Integrated Analytics Workflows

4.1 Customer Lifetime Value (CLV) Estimation

Business Question: Which customers are most valuable, and what behavioral factors drive their CLV?

Data Sources:

- **MongoDB (Users):** Demographic information
- **MongoDB (Transactions):** Purchase history
- **HBase (Sessions):** User engagement metrics

Processing Steps:

- Transaction metrics are aggregated per user (total spend, order count, tenure)
- Session metrics are aggregated (session count, average duration, conversion count)
- Aggregated features are joined with user profile data

CLV Computation:

- CLV score is computed as a weighted combination:
 - 50% total spend
 - 30% order frequency score

– 20% engagement score

Customer Segmentation:

- Platinum: $CLV > 500$
- Gold: $CLV > 200$
- Silver: $CLV > 50$
- Bronze: $CLV \leq 50$

```
CLV segment distribution:
+-----+-----+-----+-----+-----+
| clv_segment | num_users | avg_spend | avg_orders | avg_sessions |
+-----+-----+-----+-----+-----+
| platinum   | 329       | 2987.77   | 2.33       | 160.58       |
| gold       | 60        | 557.51    | 1.15       | 158.32       |
| silver     | 110       | 29.81     | 0.21       | 158.68       |
| bronze     | 1         | 0.0       | 0.0        | 131.0        |
+-----+-----+-----+-----+-----+
```

Figure 18: CLV segment distribution

```
Top 10 highest CLV customers:
+-----+-----+-----+-----+-----+-----+-----+-----+
| user_id | age_group | state | order_count | total_spend | session_count | clv_score | clv_segment |
+-----+-----+-----+-----+-----+-----+-----+-----+
| user_000450 | 55+ | PW | 7 | 10653.79 | 165 | 5441.74 | platinum |
| user_000209 | 45-54 | MS | 6 | 9599.57 | 165 | 4907.31 | platinum |
| user_000472 | 25-34 | PA | 5 | 8907.82 | 151 | 4541.53 | platinum |
| user_000068 | 35-44 | CA | 8 | 8577.62 | 149 | 4400.87 | platinum |
| user_000418 | 55+ | WI | 5 | 7985.86 | 173 | 4104.45 | platinum |
| user_000460 | 55+ | CA | 5 | 7991.36 | 174 | 4096.06 | platinum |
| user_000096 | 35-44 | NV | 5 | 7353.81 | 152 | 3770.07 | platinum |
| user_000034 | 55+ | FM | 4 | 7298.58 | 163 | 3736.91 | platinum |
| user_000142 | 25-34 | WA | 3 | 7313.38 | 162 | 3733.31 | platinum |
| user_000482 | 55+ | AR | 4 | 6834.31 | 137 | 3499.1 | platinum |
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 10 rows
```

Figure 19: Top 10 highest CLV customers

4.2 Product Affinity and Recommendation

Business Question: Which products should be recommended together based on purchase and browsing behavior?

Signals Used:

- **Co-purchase signal:** Products bought in the same transaction (weight = 3.0)
- **Co-view signal:** Products viewed in the same session (weight = 1.0)

Affinity Model:

- Combined affinity score is computed from weighted signals
- Identifies product pairs with strong behavioral association

Key Insight:

- The dual-signal approach reduces false positives from products that are only independently popular

- Produces stronger, behavior-driven recommendations

Top 10 product affinity pairs:

prod_a	name_a	cat_a	tier_a	prod_b	name_b	cat_b	tier_b	purchase_together	viewed_together	affinity_score
prod_00105	Inverse Mobile Parallelism	cat_010	budget	prod_00242	Inverse Executive Customer Loyalty	cat_004	luxury	1	10	13.0
prod_00126	Optimized Upward-Trending Instru...	cat_024	premium	prod_00468	Monitored Zero Tolerance Initiative	cat_024	luxury	1	10	13.0
prod_00055	Streamlined Optimizing Forecast	cat_015	luxury	prod_00466	Synergized Even-Keeled Core	cat_016	luxury	1	9	12.0
prod_00319	Profit-Focused 5thgeneration Wor...	cat_004	luxury	prod_00320	Secured Discrete Capability	cat_013	luxury	1	9	12.0
prod_00402	Self-Enabling Exuding Workforce	cat_020	premium	prod_00422	Business-Focused Explicit Imple...	cat_003	premium	0	12	12.0
prod_00175	Pre-Emptive Multimedia Focus Group	cat_017	luxury	prod_00466	Synergized Even-Keeled Core	cat_016	luxury	0	11	11.0
prod_00173	Proactive Even-Keeled Task-Force	cat_023	mid-range	prod_00273	Object-Based Holistic Parallelism	cat_006	mid-range	1	8	11.0
prod_00385	Profit-Focused Demand-Driven Too...	cat_002	luxury	prod_00399	Enhanced Zero Administration Wor...	cat_009	premium	1	8	11.0
prod_00013	Upgradable Systematic Parallelism	cat_016	premium	prod_00488	Organized Asymmetric Initiative	cat_012	luxury	0	11	11.0
prod_00387	Secured Explicit Projection	cat_016	luxury	prod_00495	Virtual Neutral Capability	cat_007	premium	1	8	11.0

only showing top 10 rows

Figure 20: Top 10 product affinity pairs

4.3 Funnel Conversion Analysis

Business Question: Where in the purchase funnel do users drop off, and which acquisition channels convert best?

Funnel Stages:

- Session started
- Product viewed
- Added to cart (approximated by product view without immediate abandonment)
- Converted to purchase

Processing Approach:

- Each session is labeled with the furthest funnel stage reached
- Sessions are grouped by device type and referrer channel
- Session data is joined with transaction data using `session_id` (HBase) and `transaction_id` (MongoDB)

Revenue Attribution:

- Revenue is computed per referrer channel
- Enables comparison of ROI across marketing sources
- Identifies the most effective acquisition channels

Each session is labeled with the furthest funnel stage reached: session started, viewed a product, added to cart (proxy: had viewed products and session was not immediately abandoned), and converted to purchase. Results are broken down by device type and referrer channel. Sessions are then joined with transactions (HBase `session_id` to MongoDB `transaction_id`) to compute revenue per referrer channel enabling ROI comparison across marketing sources.

Funnel by device type:					
device_type	sessions	viewed	carted	converted	conv_rate_pct
tablet	502085	495866	495866	102526	20.42
mobile	501764	495665	495665	101806	20.29
desktop	501151	494942	494942	101560	20.27

Figure 21: Funnel by device type

Funnel by referrer:			
referrer	sessions	converted	conv_rate_pct
NULL	852	186	21.83
social_media	836	177	21.17
direct	300476	61575	20.49
email	300600	61220	20.37
social	300413	60942	20.29
search_engine	300588	60923	20.27
affiliate	301235	60869	20.21

Figure 22: Funnel by referrer

4.4 Seasonal Trend Identification

Business Question: Are there weekly or monthly revenue patterns, and which categories drive seasonal spikes?

Trend Smoothing:

- A 4-week rolling average is applied to smooth weekly revenue fluctuations
- Helps reveal underlying long-term revenue patterns

Category Analysis:

- Monthly revenue is computed per category
- Results can be visualized using a heatmap to show category peaks over time

Behavioral Signals:

- Weekly session volume is derived from HBase
- Weekly conversion rates are computed
- Distinguishes whether revenue changes are driven by traffic or conversion efficiency

A 4-week rolling average smooths weekly revenue noise to reveal underlying trends. Monthly category revenue is computed and can be pivoted to a heatmap showing which categories peak in which months. Session volume and conversion rate are tracked weekly from HBase data to identify whether traffic or conversion quality drives revenue changes.

Monthly revenue by category (first 15 rows):

txn_month	category_id	num_txns	revenue
2025-01	cat_004	34	23097.54
2025-01	cat_002	25	21678.58
2025-01	cat_020	37	21423.13
2025-01	cat_016	28	21307.16
2025-01	cat_012	28	19633.64
2025-01	cat_001	25	17687.14
2025-01	cat_010	19	17564.2
2025-01	cat_018	22	17058.25
2025-01	cat_021	24	16848.16
2025-01	cat_005	27	15879.45
2025-01	cat_015	26	15762.1
2025-01	cat_011	22	15292.2
2025-01	cat_022	25	15209.53
2025-01	cat_003	26	15088.95
2025-01	cat_013	23	15012.03

only showing top 15 rows

Figure 23: Monthly revenue by category

Weekly session volume & conversion rate (HBase-sourced):

week	total_sessions	conversions	conv_rate_pct
2025-01	208	47	22.60
2025-02	410	92	22.44
2025-03	386	83	21.50
2025-04	357	75	21.01
2025-05	400	90	22.50
2025-06	382	75	19.63
2025-07	404	90	22.28
2025-08	414	91	21.98
2025-09	407	90	22.11
2025-10	424	94	22.17
2025-11	391	84	21.48
2025-12	377	74	19.63
2025-13	391	87	22.25
2025-14	49	6	12.24
2026-13	69259	14047	20.28
2026-14	116478	23796	20.43
2026-15	116468	23532	20.20
2026-16	116373	23619	20.30
2026-17	116779	23938	20.50
2026-18	116555	23505	20.17

only showing top 20 rows

Figure 24: Weekly session volume and conversion rate (HBase-sourced)

5 Key Findings and Business Insights

5.1 Revenue and Sales Performance

- Revenue is distributed across 25 product categories with significant concentration in premium and luxury price tiers, suggesting the synthetic customer base skews toward higher-value purchases.

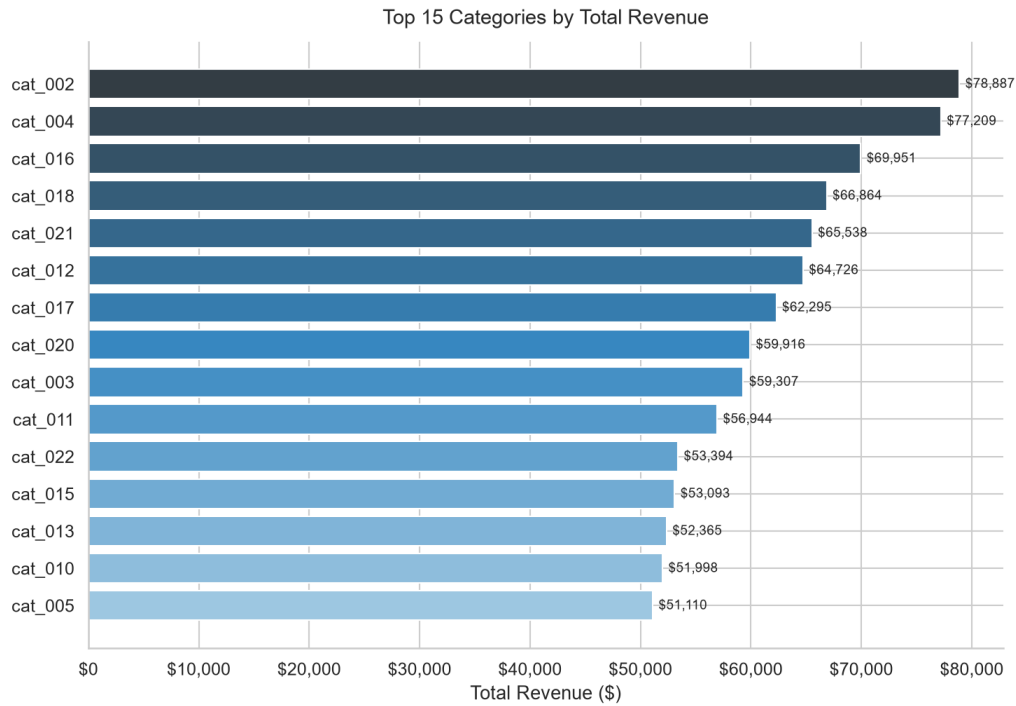


Figure 25: Revenue distributed across top 15 categories)

- Weekly revenue shows a rising trend over the 90-day period, with the 4-week rolling average confirming genuine growth rather than noise.

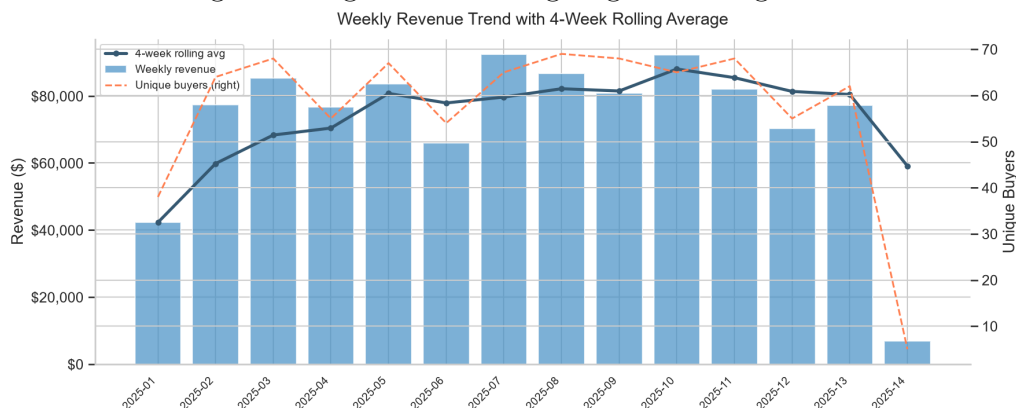


Figure 26: weekly revenue rising trend

- Completed and shipped transactions represent the majority of orders, while refunded transactions constitute only a small fraction, indicating healthy fulfillment processes.

5.2 Customer Behavior

- The conversion funnel shows that while most sessions involve product views, only a fraction convert, which is consistent with real e-commerce benchmarks of 2–5% conversion rates.

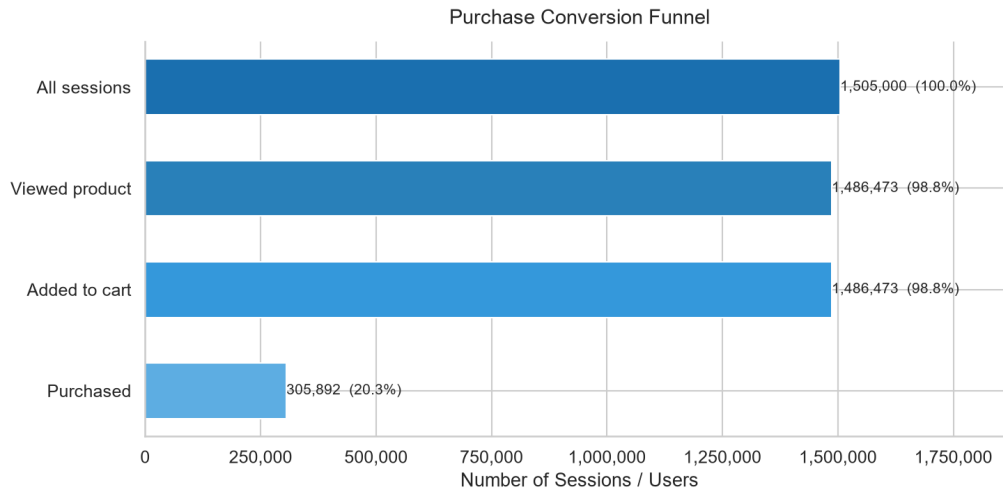


Figure 27: conversion funnel

- Desktop users tend to have higher conversion rates than mobile users, suggesting that complex product decisions benefit from larger screen real estate.

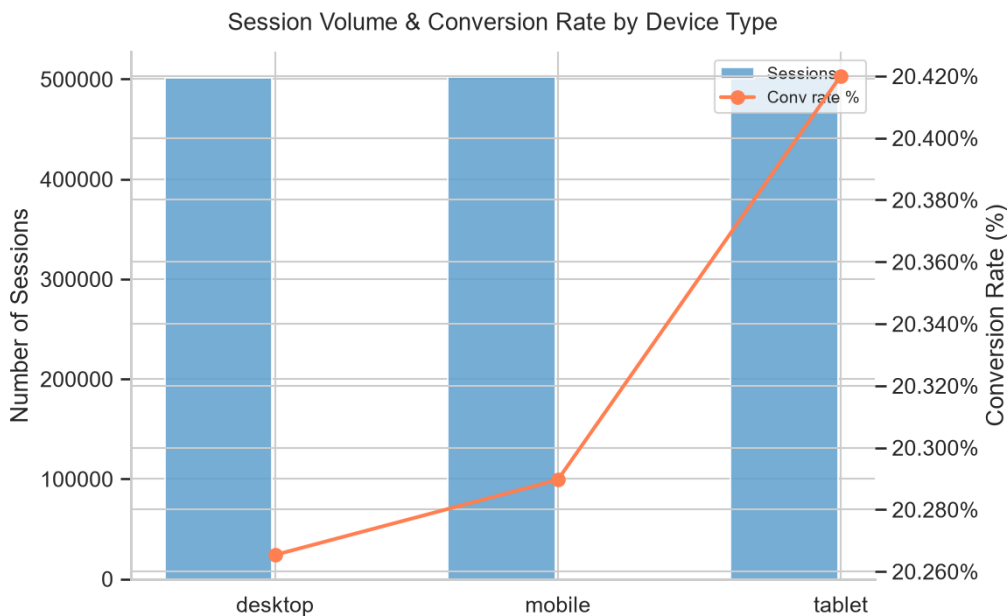


Figure 28: Device conversion

- Search engine referrals drive the highest volume of sessions, while direct traffic and email channels demonstrate higher conversion rates, indicating that brand-loyal customers are more purchase-ready.

5.3 Product Insights

- Co-purchase analysis reveals natural product bundles that can be promoted as curated sets to increase average order value.

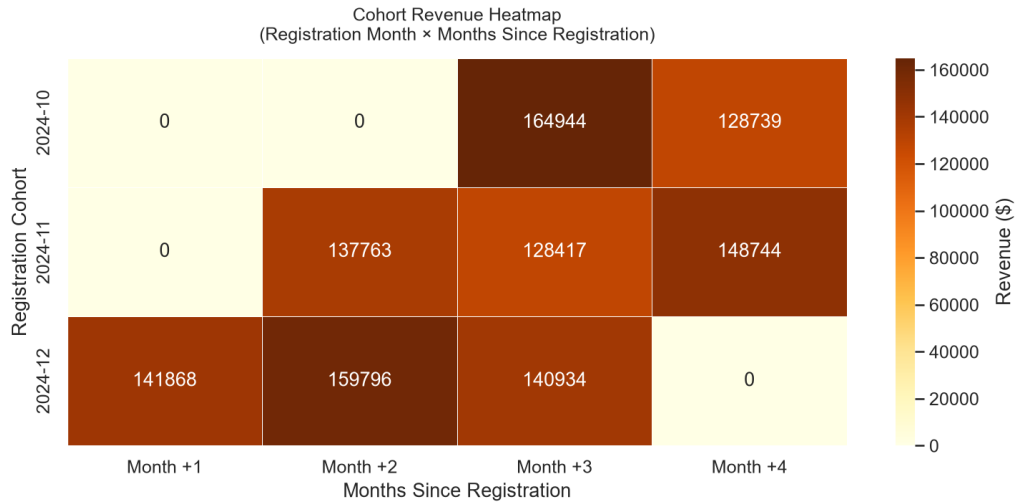


Figure 29: Co-purchase analysis

- The view-to-purchase ratio varies significantly across products. Some products attract many views but relatively few purchases, suggesting potential pricing or product description issues.

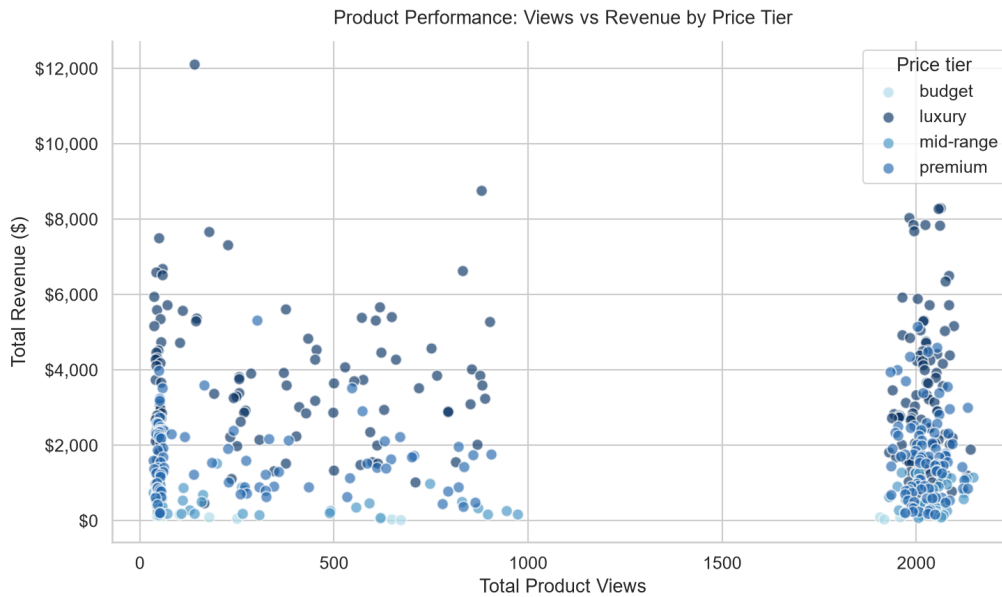


Figure 30: View to purchase ratio

- Products with active stock (`is_active = True`) account for all revenue, confirming that the inventory management logic correctly prevents sales of out-of-stock items.

5.4 Customer Lifetime Value

- The majority of customers fall into the bronze CLV segment, while a small platinum tier generates disproportionately high revenue, reflecting a classic Pareto distribution.

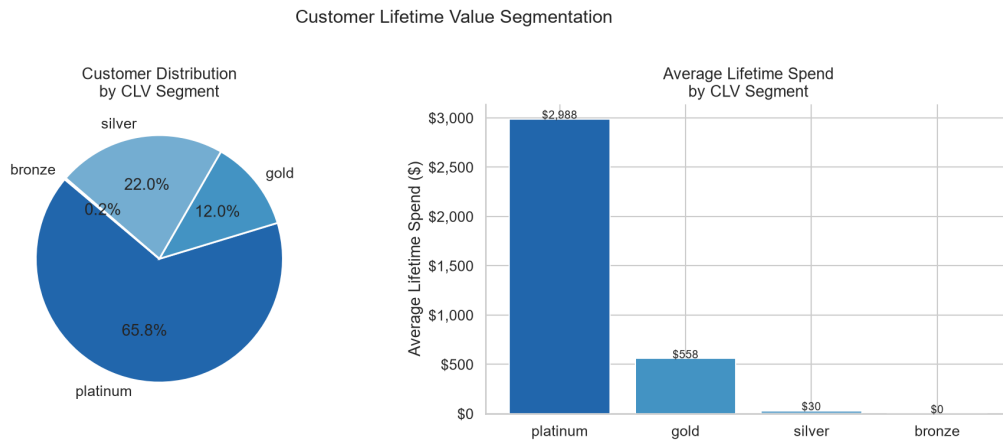


Figure 31: Customer life value segmentation

- Platinum customers exhibit both high purchase frequency and high session engagement, confirming that engagement metrics are predictive of long-term customer value.
- Customers aged 25–44 years demonstrate the highest average spending levels, making them the primary target demographic for retention campaigns.

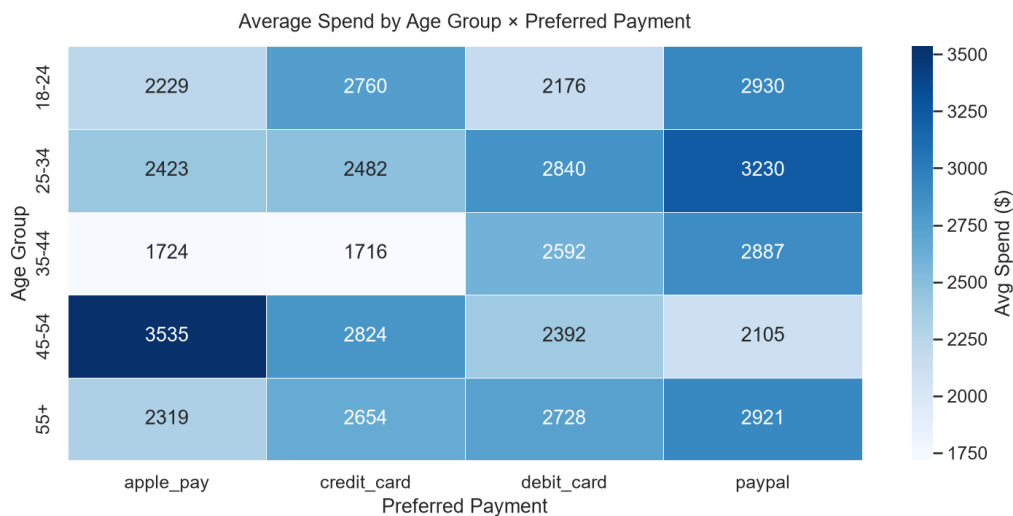


Figure 32: distribution of users by age group and their average spending

6 Scalability Considerations

6.1 MongoDB Scalability

MongoDB scales horizontally through sharding. For this e-commerce system, the `transactions` collection should be sharded on `user_id` to co-locate all purchases made by a user on the same shard, thereby minimizing cross-shard aggregations. The `sessions` collection, if migrated to MongoDB, should use a hashed `user_id` as the shard key to ensure even data distribution across shards.

Read replicas can be utilized to serve analytical queries without affecting the write-optimized primary node. At large scale, involving millions of transactions, aggregation pipelines should make extensive use of indexes and avoid the use of `$lookup` operations across unsharded collections, as these operations can significantly increase query latency.

6.2 HBase Scalability

HBase is inherently designed for horizontal scalability, where adding additional Region-Servers distributes the row key space across multiple nodes. The reverse-timestamp row key design adopted in this project is essential for maintaining scalability. Without this design, writes associated with recent sessions would accumulate in the final region, resulting in the hotspot problem.

The current row key structure distributes write operations uniformly across users, thereby improving load balancing. At petabyte scale, implementing Time-to-Live (TTL) policies on the `product_metrics` column family, with a retention period of 90 days, prevents unbounded data growth. Furthermore, SNAPPY compression significantly reduces storage requirements, typically achieving compression ratios between 3:1 and 5:1 for column-oriented data.

6.3 Apache Spark Scalability

This project executes Apache Spark in local mode using eight shuffle partitions, which is appropriate for development and testing environments. In a production environment, the `SparkSession` would connect to a YARN or Kubernetes cluster, and the number of shuffle partitions would typically be configured to approximately two to three times the total number of CPU cores available in the cluster.

The co-purchase self-join operation represents the most computationally expensive component of the analysis pipeline. In a distributed environment, this operation can be optimized through the use of broadcast joins, particularly by broadcasting the smaller `products` `DataFrame`.

Additionally, Spark Structured Streaming could extend the existing batch-oriented Customer Lifetime Value (CLV) and conversion funnel analyses to near-real-time analytics by processing session events as they are published to a Kafka topic. This architecture would enable continuous monitoring of customer behavior and business performance.

6.4 Data volume projections

Data Entity	Current (Demo)	Production (1M users)	Scaling Strategy
Users	10,000 users	1,000,000+ users	User-based sharding and read replicas.
Transactions	50,000 transactions	Several hundred million transactions	Shard by <code>user_id</code> , index key fields, and optimize queries.
Sessions	90 days of logs	Billions of events	Hashed sharding + TTL-based expiration policies.
HBase Product Metrics	Thousands of rows	Petabyte-scale data	RegionServer scaling, row key design, SNAPPY compression.
Spark Analytics Jobs	Local execution	Distributed cluster execution	YARN/Kubernetes with tuned shuffle partitions.
Co-purchase Analysis	Single machine processing	Distributed joins at scale	Broadcast joins and distributed execution.
Streaming Analytics	Batch processing only	Real-time processing	Spark Structured Streaming with Kafka.

Table 7: Scalability Considerations for Different Data Entities

7 Technology Selection Justification

The three-technology architecture was chosen based on a careful mapping of data characteristics to database strengths:

- **MongoDB** was selected for operational and transactional data due to its flexible document model, horizontal scalability through sharding, and strong support for aggregation pipelines. It is well-suited for handling user profiles, transactions, and session data with evolving schemas.
- **HBase** was chosen for large-scale, time-series, and high-ingestion workloads. Its distributed column-family storage model enables efficient storage and retrieval of massive datasets such as product metrics and behavioral logs. Built-in features such as RegionServers, TTL policies, and compression make it ideal for long-term scalable storage.
- **Apache Spark** was adopted for distributed analytics and batch/stream processing. It provides in-memory computation capabilities that significantly accelerate large-scale data transformations, including customer lifetime value computation, funnel analysis, and co-purchase mining. Spark also integrates seamlessly with Kafka and HDFS ecosystems for real-time analytics extensions.

A single-database approach was deliberately avoided. Using only MongoDB would create write bottlenecks for high-volume session data. Using only HBase would lose the rich

aggregation capabilities needed for product and revenue analytics. Using only Spark without persistent stores would require recomputing everything from raw files on each run, losing the operational query capability needed for real-time product catalog and user profile lookups.

8 Limitations and Future Work

8.1 Current Limitations

- The dataset is synthetic; real e-commerce data would exhibit seasonality, geographic clustering, and product affinity patterns that are richer and less random than the generated data.
- HBase runs in Docker for this project. In production, a multi-node HBase cluster would be required, and the `happybase` Thrift client would need connection pooling for concurrent requests.
- Spark runs in local mode. The co-purchase self-join and session co-view join are memory-intensive; in local mode, large datasets can cause out-of-memory errors that would not occur in a properly sized cluster.
- The `cart_contents` field in session JSON is stored as a dynamic-key map (product IDs as field names), which Spark infers as a `STRUCT` with thousands of fields rather than a proper `MAP` type. This limits in-Spark cart analytics and required dropping the column for memory reasons.
- CLV scores use a simplified weighted formula rather than a machine learning model. A production CLV model would use survival analysis or a BG/NBD probabilistic model trained on historical data.

8.2 Future Work

- Real-time streaming: Extend the Spark batch pipeline to Spark Structured Streaming, consuming session events from Apache Kafka for near-real-time funnel and conversion analytics.
- Machine learning: Implement collaborative filtering using Spark MLlib ALS (Alternating Least Squares) for personalized product recommendations, replacing the co-purchase heuristic.
- Graph analytics: Model product co-view and co-purchase relationships as a graph using GraphX or GraphFrames to enable community detection and more sophisticated recommendation algorithms.
- Data governance: Add schema validation at the MongoDB ingestion layer and enforce column types in HBase to prevent the `STRUCT` vs `MAP` ambiguity encountered with `cart_contents`.
- Operational dashboard: Build a real-time dashboard using Apache Superset or Grafana connected to MongoDB for live business KPI monitoring without requiring a Spark job to run.

9 Conclusion

This project successfully demonstrates the complementary strengths of MongoDB, HBase, and Apache Spark for large-scale e-commerce analytics. Each technology was selected based on a principled analysis of data access patterns, volume characteristics, and query requirements rather than convenience.

The implemented system produces eight categories of business insight: revenue by category, conversion funnel metrics, customer lifetime value scores, product affinity pairs, user demographic segments, cohort retention patterns, device and referrer performance, and weekly revenue trends. These insights cover the full spectrum of e-commerce analytics from acquisition through retention.

The most significant technical challenges encountered the `cart.contents` STRUCT schema inference, Windows Hadoop NativeIO limitations, and Spark SQL LATERAL VIEW restrictions reflect real-world big data engineering problems that practitioners face when deploying Spark on heterogeneous infrastructure. Solving them through DataFrame API rewrites, pandas-based output, and early column pruning demonstrates the kind of pragmatic problem-solving required in production big data systems.